Assignment -7

Q-1

Create a Student class with the following attributes: name, age, and student_id.
The constructor should initialize these attributes.

Add a method display_info() that prints the student's details in a user-friendly format.

Add a method is_eligible_to_vote() that returns True if the student's age is 18 or older,

and False otherwise.
Create a student object and test all the methods

Welcome | Ass7Q1.py

Ass7Q1.py > ...

```python
class Student:
    def __init__(self,name, age, student_id):
        self.name=name
        self.age=age
        self.student_id=student_id

    def display_info(self):
        print("Student Details")
        print("name :",self.name)
        print("age :",self.age)
        print("Student id :",self.student_id)

    def is_eligible_to_vote(self):
        return self.age >=18

#Creating the student object

student1=Student("Shubham", 20, "101")

student1.display_info()

if student1.is_eligible_to_vote():
    print("Eligible to vote: True")

else:
    print("Eligible to vote : False")
```

Problems | Output | Debug Console | Terminal | Ports

```
[Running] python -u "c:\Users\Shubham M. Dande\python\Ass7Q1.py"
  File "c:\Users\Shubham M. Dande\python\Ass7Q1.py", line 25
SyntaxError: expected ':'

[Done] exited with code=1 in 0.213 seconds

[Running] python -u "c:\Users\Shubham M. Dande\python\Ass7Q1.py"
Student Details
name : Shubham
age : 20
Student id : 101
Eligible to vote: True

[Done] exited with code=0 in 0.207 seconds
```

Q-2

- Create a base class Vehicle with attributes: make, model, and year. The constructor should initialize these.
- Add a method display_info() that prints the basic vehicle information.
- Create a child class Car that inherits from Vehicle.
- The Car class should have an additional attribute num_doors. The constructor should take make, model, year, and num_doors.
- Override the display_info() method in the Car class to also include the number of doors.
- Create instances of both Vehicle and Car to demonstrate inheritance and method overriding.

Welcome | Ass7Q1.py | ASS7Q2.py

ASS7Q2.py > ...

```python
class Vehicle:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year

    def display_info(self):
        print("Vehicle Information")
        print("Make:", self.make)
        print("Model:", self.model)
        print("Year:", self.year)


class Car(Vehicle):
    def __init__(self, make, model, year, num_doors):
        super().__init__(make, model, year)
        self.num_doors = num_doors

    # Method overriding
    def display_info(self):
        print("Car Information")
        print("Make:", self.make)
        print("Model:", self.model)
        print("Year:", self.year)
        print("Number of Doors:", self.num_doors)


# Create Vehicle object
vehicle1 = Vehicle("Toyota", "Corolla", 2020)
vehicle1.display_info()

print()

# Create Car object
car1 = Car("Honda", "City", 2023, 4)
car1.display_info()
```

Problems | Output | ... | Filter (e.g. text, !excludeText, t... | Code

```
[Running] python -u "c:\Users\Shubham M. Dande\python\ASS7Q2.py"
Vehicle Information
Make: Toyota
Model: Corolla
Year: 2020

Car Information
Make: Honda
Model: City
Year: 2023
Number of Doors: 4

[Done] exited with code=0 in 0.178 seconds
```

Q.3

- Employee Management System:
- Create a base class Employee with attributes name, employee_id, and salary.
- Create a derived class Manager that inherits from Employee and has an additional attribute for a list of employees they manage.
- Add methods to both classes, such as a method in Manager to add an employee to their managed list.

Welcome | Ass7Q1.py | ASS7Q2.py | ASS7Q3.py

ASS7Q3.py > ...

```python
class Employee:
    def __init__(self, name, employee_id, salary):
        self.name = name
        self.employee_id = employee_id
        self.salary = salary

    def display_info(self):
        print("Employee Name:", self.name)
        print("Employee ID:", self.employee_id)
        print("Salary:", self.salary)


class Manager(Employee):
    def __init__(self, name, employee_id, salary):
        super().__init__(name, employee_id, salary)
        self.managed_employees = []

    def add_employee(self, employee):
        self.managed_employees.append(employee)

    def display_managed_employees(self):
        print("Employees managed by", self.name)

        for employee in self.managed_employees:
            print(employee.name, "-", employee.employee_id)


# Create employees
employee1 = Employee("Rahul", "E101", 30000)
employee2 = Employee("Amit", "E102", 35000)

# Create manager
manager1 = Manager("Priya", "M101", 60000)

# Add employees to manager
manager1.add_employee(employee1)
manager1.add_employee(employee2)

# Display manager information
manager1.display_info()

print()

# Display managed employees
manager1.display_managed_employees()
```

```
[Running] python -u "c:\Users\Shubham M. Dande\python\ASS7Q3.py"
Employee Name: Priya
Employee ID: M101
Salary: 60000

Employees managed by Priya
Rahul - E101
Amit - E102

[Done] exited with code=0 in 0.239 seconds
```

Q.4

Library Management System:

- Create a Book class with attributes like title, author, and is_available.
- Create a Library class that has a list of Book objects.
- Add methods to the Library class to:
  - add_book(book): Adds a new book to the library's collection.
  - borrow_book(title): Finds a book by title and changes its availability status.
  - return_book(title): Changes a book's availability status back to available.
  - display_available_books(): Prints the titles and authors of all available books.

Welcome | ASS7Q4.py

ASS7Q4.py > ...

```python
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
        self.is_available = True


class Library:
    def __init__(self):
        self.books = []

    def add_book(self, book):
        self.books.append(book)
        print("Book added:", book.title)

    def borrow_book(self, title):
        for book in self.books:
            if book.title.lower() == title.lower():
                if book.is_available:
                    book.is_available = False
                    print("Book borrowed:", book.title)
                else:
                    print("Book is already borrowed.")
                return

        print("Book not found.")

    def return_book(self, title):
        for book in self.books:
            if book.title.lower() == title.lower():
                book.is_available = True
                print("Book returned:", book.title)
                return

        print("Book not found.")

    def display_available_books(self):
        print("\nAvailable Books:")

        for book in self.books:
            if book.is_available:
                print(book.title, "-", book.author)


# Create library
library = Library()

# Create books
book1 = Book("Python Programming", "John Smith")
book2 = Book("Data Structures", "Robert Brown")
book3 = Book("Computer Networks", "Andrew Tanenbaum")
```

```
52
53  # Add books
54  library.add_book(book1)
55  library.add_book(book2)
56  library.add_book(book3)
57
58  # Display available books
59  library.display_available_books()
60
61  # Borrow a book
62  library.borrow_book("Python Programming")
63
64  # Display available books
65  library.display_available_books()
66
67  # Return the book
68  library.return_book("Python Programming")
69
70  # Display available books
71  library.display_available_books()
```

```
[Running] python -u "c:\Users\Shubham M. Dande\python\ASS7Q4.py"
Book added: Python Programming
Book added: Data Structures
Book added: Computer Networks

Available Books:
Python Programming - John Smith
Data Structures - Robert Brown
Computer Networks - Andrew Tanenbaum
Book borrowed: Python Programming

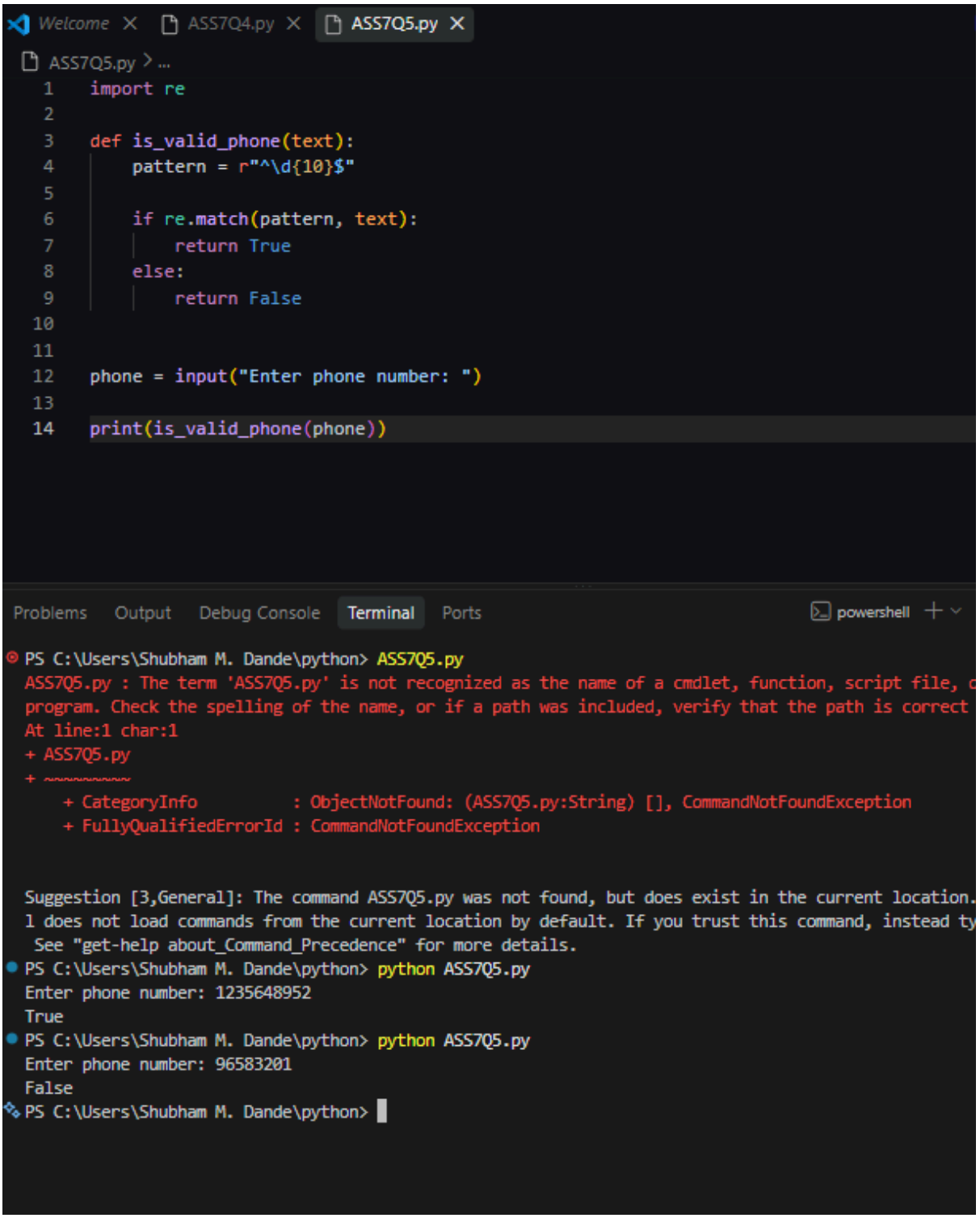
Available Books:
Data Structures - Robert Brown
Computer Networks - Andrew Tanenbaum
Book returned: Python Programming

Available Books:
Python Programming - John Smith
Data Structures - Robert Brown
Computer Networks - Andrew Tanenbaum

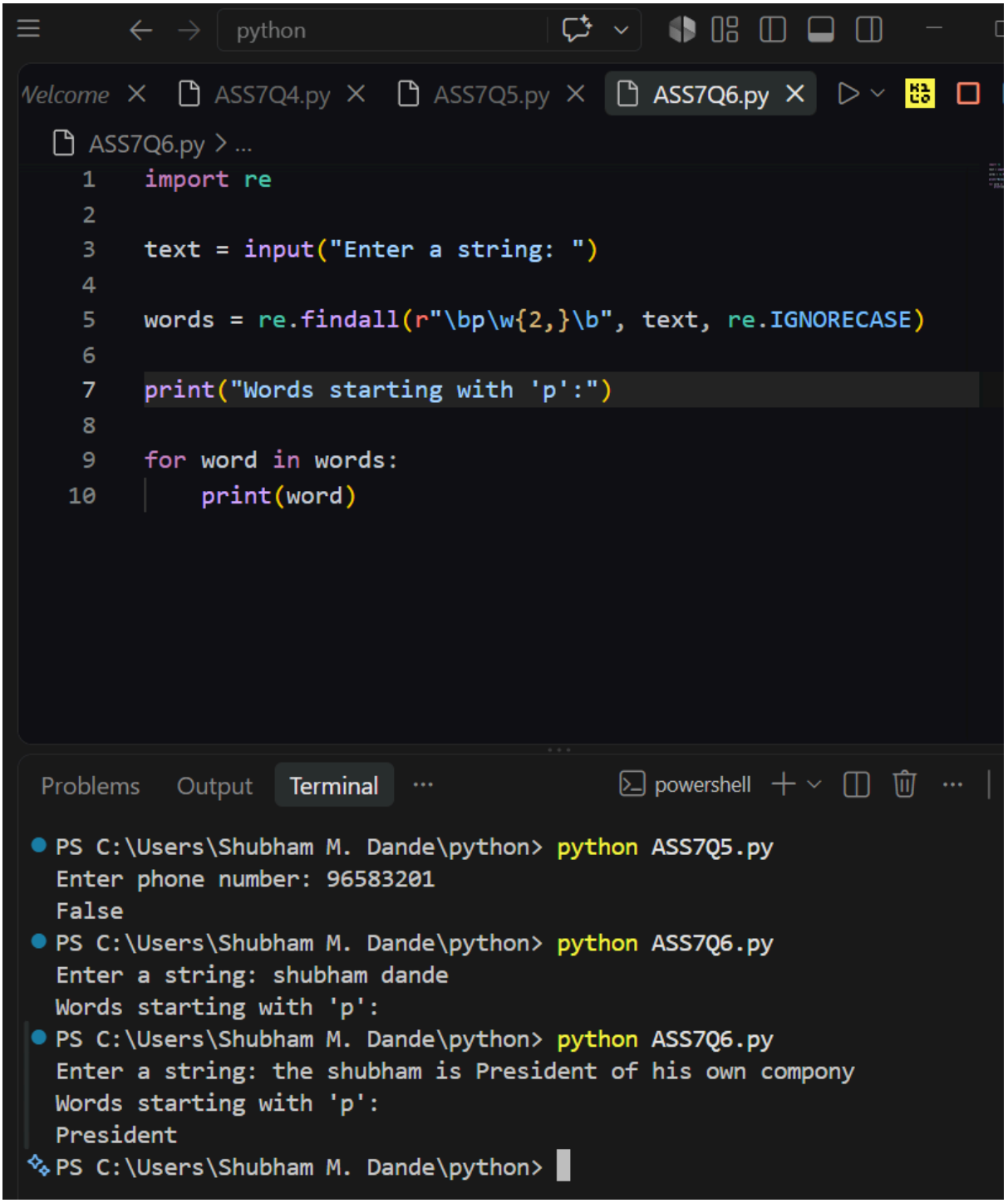
[Done] exited with code=0 in 0.309 seconds
```

Write a Python function is_valid_phone(text) that takes a string and returns True if it is a valid 10-digit phone number, and False otherwise. A valid phone number should consist of

exactly 10 digits.



Write a Python program to find all words in a string that start with the letter 'p' (case-insensitive) and are at least 3 characters long.



Write a Python function find_dates(text) to find the first occurrence of a date in the format MM-DD-YYYY and prints the full match.

ASS7Q7.py > ...

```python
import re

def find_dates(text):
    pattern = r"\b\d{2}-\d{2}-\d{4}\b"

    match = re.search(pattern, text)

    if match:
        print("Date found:", match.group())
    else:
        print("No date found.")


text = input("Enter a string: ")
find_dates(text)
```

Problems Output Terminal ··· powershell

```
PS C:\Users\Shubham M. Dande\python> python ASS7Q6.py
PS C:\Users\Shubham M. Dande\python> python ASS7Q6.py
Enter a string: the shubham is President of his own compony
Words starting with 'p':
President
PS C:\Users\Shubham M. Dande\python> python ASS7Q7.py
Enter a string: shubham dande
No date found.
PS C:\Users\Shubham M. Dande\python> python ASS7Q7.py
Enter a string: my birthday is on 12-04-3924
Date found: 12-04-3924
PS C:\Users\Shubham M. Dande\python>
```

Write a Python function is_strong_password(password) that uses regex to check if a password meets the following criteria:

- At least 8 characters long.
- Contains at least one uppercase letter.
- Contains at least one lowercase letter.
- Contains at least one digit.
- Contains at least one special character (e.g., !, @, #, $, %, ^, &, *).

```python
import re

def is_strong_password(password):
    pattern = r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[!@#$%^&*]).{8,}$"

    if re.match(pattern, password):
        return True
    else:
        return False


password = input("Enter password: ")

print(is_strong_password(password))
```

```
Problems  Output  Debug Console  Terminal  Ports  powershell

PS C:\Users\Shubham M. Dande\python> python ASS7Q7.py
No date found.
PS C:\Users\Shubham M. Dande\python> python ASS7Q7.py
Enter a string: my birthday is on 12-04-3924
Date found: 12-04-3924
PS C:\Users\Shubham M. Dande\python> python ASS7Q8.py
Enter password: Shubham@12345
True
PS C:\Users\Shubham M. Dande\python> python ASS7Q8.py
Enter password: shubham
False
```

Write a Python function clean_string(text) that takes a string and performs the following cleaning tasks using regex:

- Replace multiple spaces with a single space.
- Remove leading and trailing whitespace.
- Capitalize the first letter of each sentence.

ASS7Q9.py > ...

```python
import re

def clean_string(text):
    # Replace multiple spaces with one space
    text = re.sub(r"\s+", " ", text)

    # Remove leading and trailing whitespace
    text = text.strip()

    # Capitalize first letter of each sentence
    text = re.sub(
        r"(^|[.!?]\s+)([a-z])",
        lambda m: m.group(1) + m.group(2).upper(),
        text
    )

    return text


text = input("Enter a string: ")

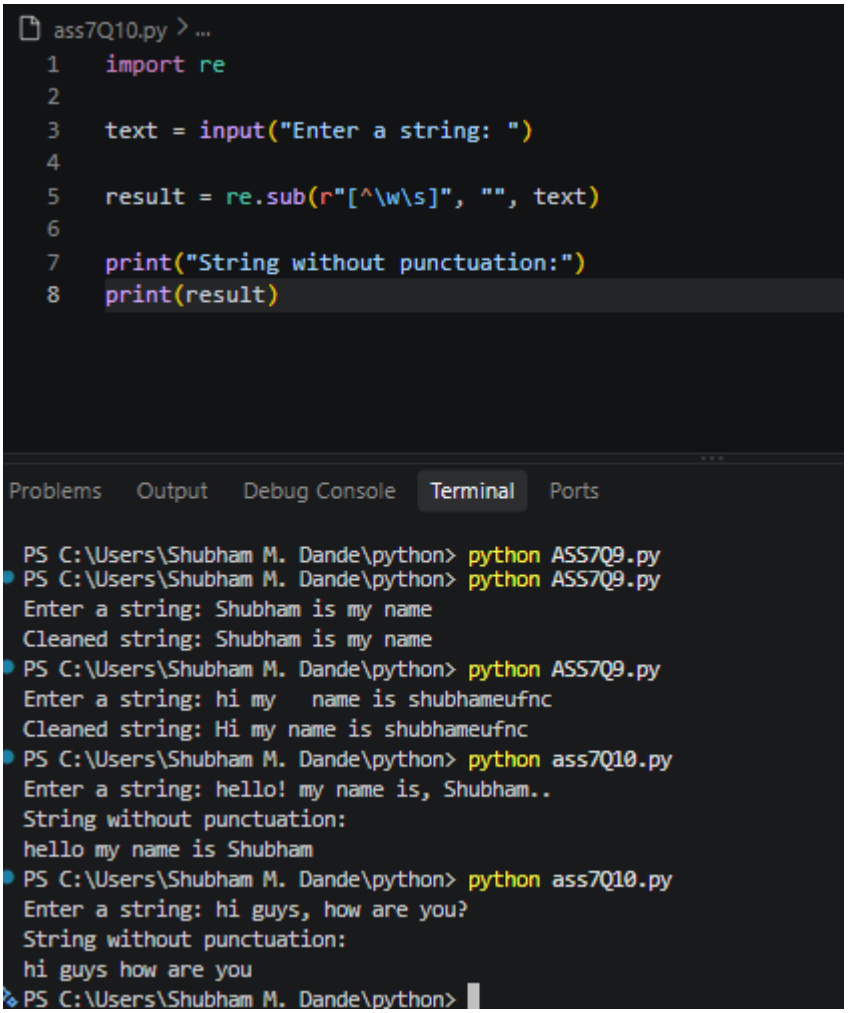
result = clean_string(text)

print("Cleaned string:", result)
```

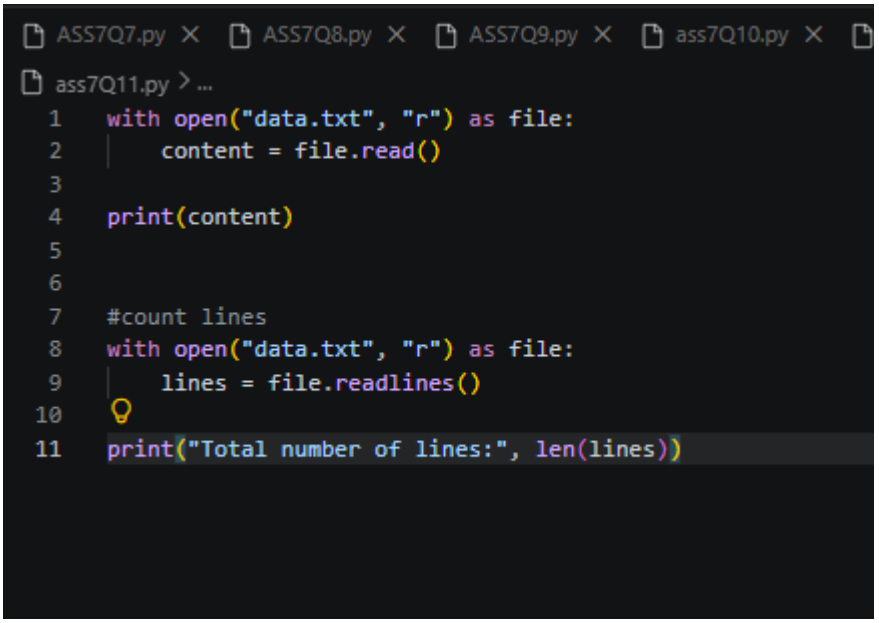
Problems Output Debug Console Terminal Ports powers

```
PS C:\Users\Shubham M. Dande\python> ASS7Q9.py
9.py". See "get-help about_Command_Precedence" for more details.
PS C:\Users\Shubham M. Dande\python> python ASS7Q9.py
Enter a string: Shubham
Cleaned string: Shubham
PS C:\Users\Shubham M. Dande\python> python ASS7Q9.py
Enter a string: Shubham is my name
Cleaned string: Shubham is my name
PS C:\Users\Shubham M. Dande\python> python ASS7Q9.py
Enter a string: hi my   name is shubhameufnc
Cleaned string: Hi my name is shubhameufnc
PS C:\Users\Shubham M. Dande\python>
```

Write a Python program that takes a string and replace all punctuation marks (e.g., ,, ., !, ?) with an empty string.

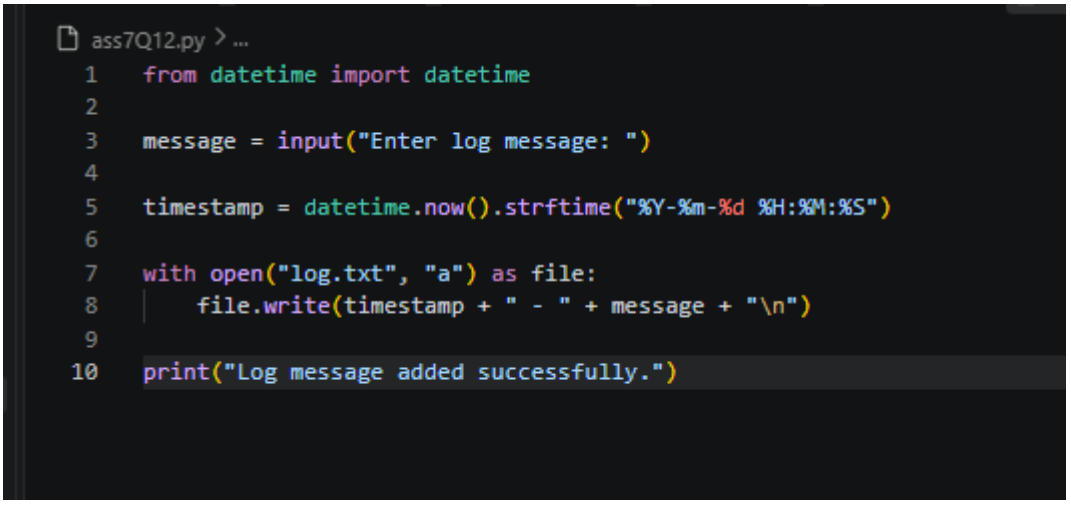


- Read and Print a File:
- Create a text file named data.txt with a few lines of text.
- Write a Python script that opens data.txt in read mode, reads the entire content, and prints it to the console.
- Make sure to use a with statement to handle the file automatically.
- Count Lines in a File:
- Use the data.txt file from the previous question.
- Write a Python program that counts the number of lines in the file and prints the total count.




```
[Running] python -u "c:\Users\Shubham M. Dande\python\ass7Q11.py"
Hello, this is the first line.
This is the second line.
Python file handling is easy.
Total number of lines: 3
```

- Append to a File:
- Create a file named log.txt.
- Write a Python program that prompts the user to enter a log message.
- The program should then append this message, along with a timestamp, to the end of log.txt on a new line.




```
PS C:\Users\Shubham M. Dande\python> python ass7Q12.py
Enter log message: buddha is the gretest person the human have
Log message added successfully.
PS C:\Users\Shubham M. Dande\python>
```

- Write a Simple CSV File:

- Write a Python script that takes a list of lists, where each inner list represents a row of data (e.g., [['Name', 'Age'], ['Alice', 30], ['John', 25]]).
- The script should write this data to a new CSV file named people.csv.
- Use the csv module for this task.

ass7Q13.py > file

```python
import csv

data = [
    ["Name", "Age"],
    ["Alice", 30],
    ["John", 25]
]

with open("people.csv", "w", newline="") as file:
    writer = csv.writer(file)

    writer.writerows(data)

print("CSV file created successfully.")
```

```
[Running] python -u "c:\Users\Shubham M. Dande\python\ass7Q13.py"
CSV file created successfully.

[Done] exited with code=0 in 0.267 seconds
```

people.csv

people.csv > data

```
Name,Age
Alice,30
John,25
```

Assignment 8

Q1.
Write a Python program that accepts a sentence from the user and performs the following:

1. Count the number of vowels and consonants.
2. Count the number of digits and special characters.
3. Find the longest word in the sentence.
4. Display all words having more than 5 characters.
5. Check whether the sentence is a palindrome after removing spaces and converting it to lowercase.

people.csv | Q1.py

ASSIGNMENT8 > Q1.py > ...

```python
sentence = input("Enter a sentence: ")

vowels = 0
consonants = 0
digits = 0
special = 0

# Count vowels, consonants, digits and special characters
for ch in sentence:
    if ch.isalpha():
        if ch.lower() in "aeiou":
            vowels += 1
        else:
            consonants += 1
    elif ch.isdigit():
        digits += 1
    elif ch != " ":
        special += 1

print("Number of vowels:", vowels)
print("Number of consonants:", consonants)
print("Number of digits:", digits)
print("Number of special characters:", special)

# Find longest word
words = sentence.split()

longest_word = ""

for word in words:
    if len(word) > len(longest_word):
        longest_word = word

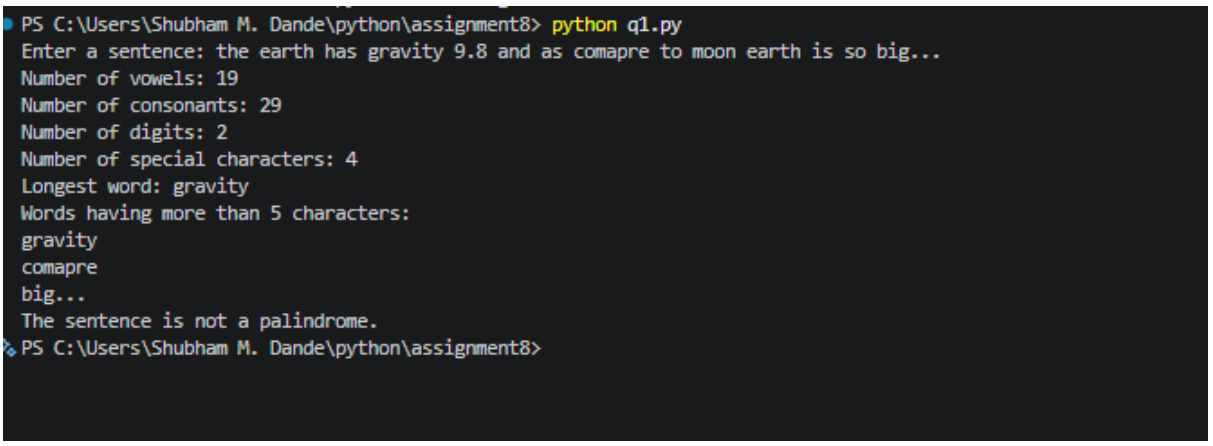
print("Longest word:", longest_word)

# Words having more than 5 characters
print("Words having more than 5 characters:")

for word in words:
    if len(word) > 5:
        print(word)

# Check palindrome
cleaned = sentence.replace(" ", "").lower()

if cleaned == cleaned[::-1]:
    print("The sentence is a palindrome.")
else:
    print("The sentence is not a palindrome.")
```



Q2.
Write a Python program to accept marks of n students into a list.
Perform the following operations:

1. Display all marks.
2. Find the highest and lowest marks without using max() or min().
3. Calculate the average marks.
4. Display students who scored above the average.
5. Count the number of students in each category:
   - 75 and above – Distinction
   - 60–74 – First Class
   - 50–59 – Second Class
   - Below 50 – Fail
6. Display the list in reverse order using a loop.

people.csv | Q1.py | Q2.py

ASSIGNMENT8 > Q2.py > ...

```python
n = int(input("Enter number of students: "))

marks = []

# Accept marks
for i in range(n):
    mark = float(input("Enter marks of student " + str(i + 1) + ": "))
    marks.append(mark)

# Display all marks
print("\nAll Marks:")
for mark in marks:
    print(mark)

# Find highest and lowest without max() and min()
highest = marks[0]
lowest = marks[0]

for mark in marks:
    if mark > highest:
        highest = mark

    if mark < lowest:
        lowest = mark

print("Highest marks:", highest)
print("Lowest marks:", lowest)

# Calculate average
total = 0

for mark in marks:
    total += mark

average = total / n

print("Average marks:", average)

# Students above average
print("\nStudents who scored above average:")

for i in range(n):
    if marks[i] > average:
        print("Student", i + 1, ":", marks[i])

# Count categories
distinction = 0
first_class = 0
second_class = 0
fail = 0
```

```
52  for mark in marks:
53      if mark >= 75:
54          distinction += 1
55      elif mark >= 60:
56          first_class += 1
57      elif mark >= 50:
58          second_class += 1
59      else:
60          fail += 1
61
62  print("\nCategories:")
63  print("Distinction:", distinction)
64  print("First Class:", first_class)
65  print("Second Class:", second_class)
66  print("Fail:", fail)
67
68  # Reverse list using loop
69  print("\nMarks in reverse order:")
70
71  for i in range(n - 1, -1, -1):
72      print(marks[i])
```

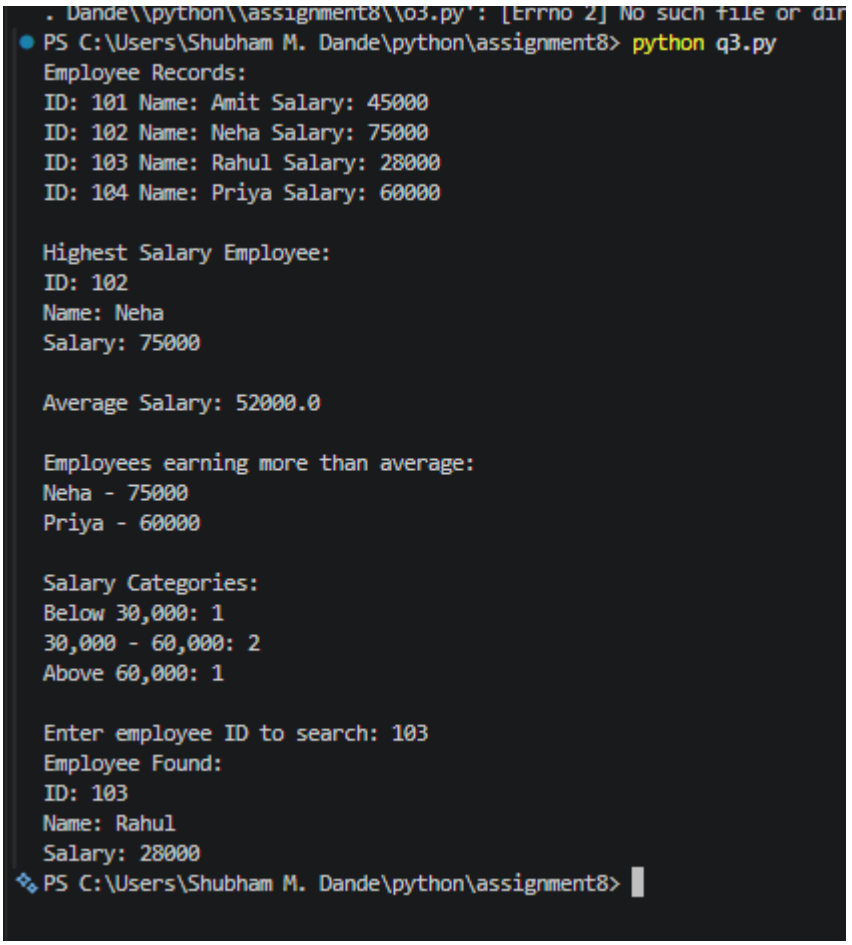
```
PS C:\Users\Shubham M. Dande\python\assignment8> python q2.py
Enter number of students: 3
Enter marks of student 1: 89
Enter marks of student 2: 54
Enter marks of student 3: 86

All Marks:
89.0
54.0
86.0
Highest marks: 89.0
Lowest marks: 54.0
Average marks: 76.33333333333333

Students who scored above average:
Student 1 : 89.0
Student 3 : 86.0

Categories:
Distinction: 2
First Class: 0
Second Class: 1
Fail: 0

Marks in reverse order:
86.0
54.0
89.0
PS C:\Users\Shubham M. Dande\python\assignment8>
```



Q3.
Create a tuple containing employee records in the form:
(employee_id, employee_name, salary)
Write a program to:

1. Display all employees.
2. Find the employee receiving the highest salary.
3. Calculate the average salary.
4. Display employees whose salary is greater than the average.
5. Count employees based on salary categories:
   - o Below 30,000
   - o 30,000–60,000
   - o Above 60,000
6. Search for an employee by employee ID.

people.csv | Q1.py | Q2.py | q3.py

ASSIGNMENT8 > q3.py > ...

```python
employees = (
    (101, "Amit", 45000),
    (102, "Neha", 75000),
    (103, "Rahul", 28000),
    (104, "Priya", 60000)
)

# 1. Display all employees
print("Employee Records:")

for employee in employees:
    print("ID:", employee[0],
          "Name:", employee[1],
          "Salary:", employee[2])

# 2. Find employee with highest salary
highest = employees[0]

for employee in employees:
    if employee[2] > highest[2]:
        highest = employee

print("\nHighest Salary Employee:")
print("ID:", highest[0])
print("Name:", highest[1])
print("Salary:", highest[2])

# 3. Calculate average salary
total = 0

for employee in employees:
    total += employee[2]

average = total / len(employees)

print("\nAverage Salary:", average)

# 4. Employees earning more than average
print("\nEmployees earning more than average:")

for employee in employees:
    if employee[2] > average:
        print(employee[1], "-", employee[2])
```

```
        print(employee[1], "-", employee[2])

# 5. Count salary categories
below_30000 = 0
between_30000_60000 = 0
above_60000 = 0

for employee in employees:
    salary = employee[2]

    if salary < 30000:
        below_30000 += 1
    elif salary <= 60000:
        between_30000_60000 += 1
    else:
        above_60000 += 1

print("\nSalary Categories:")
print("Below 30,000:", below_30000)
print("30,000 - 60,000:", between_30000_60000)
print("Above 60,000:", above_60000)

# 6. Search employee by ID
search_id = int(input("\nEnter employee ID to search: "))

found = False

for employee in employees:
    if employee[0] == search_id:
        print("Employee Found:")
        print("ID:", employee[0])
        print("Name:", employee[1])
        print("Salary:", employee[2])
        found = True
        break

if not found:
    print("Employee not found.")
```

Q4.
Accept two lists of integers from the user and convert them into sets.
Write a program to:

1. Display unique elements from both lists.
2. Find the union.
3. Find the intersection.
4. Find elements present only in the first list.
5. Find elements present only in the second list.
6. Display common even numbers.

7. Display common odd numbers.

people.csv | Q1.py | q4.py | Q2.py | q3.py

ASSIGNMENT8 > q4.py > ...

```python
# Accept first list
n1 = int(input("Enter number of elements in first list: "))

list1 = []

for i in range(n1):
    value = int(input("Enter element: "))
    list1.append(value)

# Accept second list
n2 = int(input("Enter number of elements in second list: "))

list2 = []

for i in range(n2):
    value = int(input("Enter element: "))
    list2.append(value)

# Convert lists into sets
set1 = set(list1)
set2 = set(list2)

# 1. Unique elements from both lists
print("\nUnique elements from both lists:")
print(set1 | set2)

# 2. Union
print("\nUnion:")
print(set1.union(set2))

# 3. Intersection
print("\nIntersection:")
print(set1.intersection(set2))

# 4. Only in first list
print("\nElements only in first list:")
print(set1 - set2)
```

```python

# 5. Only in second list
print("\nElements only in second list:")
print(set2 - set1)

# 6. Common even numbers
common = set1.intersection(set2)

print("\nCommon even numbers:")

for num in common:
    if num % 2 == 0:
        print(num)

# 7. Common odd numbers
print("\nCommon odd numbers:")

for num in common:
    if num % 2 != 0:
        print(num)
```

```
PS C:\Users\Shubham M. Dande\python\assignment8> python q4.py
Enter number of elements in first list: 2
Enter element: 12
Enter element: 69
Enter number of elements in second list: 3
Enter element: 89
Enter element: 54
Enter element: 25

Unique elements from both lists:
{69, 54, 89, 12, 25}

Union:
{69, 54, 89, 12, 25}

Intersection:
set()

Elements only in first list:
{12, 69}

Elements only in second list:
{89, 54, 25}

Common even numbers:

Common odd numbers:
PS C:\Users\Shubham M. Dande\python\assignment8>
```

Q5.
Create a dictionary containing student names and their marks in three subjects.
Example:

```
students = {
    "Amit": [75, 82, 68],
    "Neha": [91, 88, 95],
    "Rahul": [45, 52, 48]
}
```

Write a program to:

1. Calculate total and average marks for each student.
2. Assign grades using conditional statements.
3. Display students who have passed in all subjects.
4. Display the student with the highest average.
5. Count the number of students in each grade.
6. Search for a student by name.

ASSIGNMENT8 > Q5.py > ...

```python
students = {
    "Amit": [75, 82, 68],
    "Neha": [91, 88, 95],
    "Rahul": [45, 52, 48]
}

# Store grade counts
grade_A = 0
grade_B = 0
grade_C = 0
grade_D = 0
grade_F = 0

highest_student = ""
highest_average = 0

print("Student Details:\n")

for name, marks in students.items():

    # Calculate total
    total = 0

    for mark in marks:
        total += mark

    # Calculate average
    average = total / len(marks)
```

```
        # Assign grade
        if average >= 90:
            grade = "A"
            grade_A += 1
        elif average >= 75:
            grade = "B"
            grade_B += 1
        elif average >= 60:
            grade = "C"
            grade_C += 1
        elif average >= 50:
            grade = "D"
            grade_D += 1
        else:
            grade = "F"
            grade_F += 1

        print("Name:", name)
        print("Marks:", marks)
        print("Total:", total)
        print("Average:", average)
        print("Grade:", grade)
        print()

        # Find highest average
        if average > highest_average:
            highest_average = average
            highest_student = name

    # 3. Students passed in all subjects
    print("Students who passed in all subjects:")

    for name, marks in students.items():

        passed = True

        for mark in marks:
            if mark < 40:
                passed = False
                break

```

```
        if passed:
            print(name)

    # 4. Student with highest average
    print("\nStudent with highest average:")
    print(highest_student)
    print("Average:", highest_average)

    # 5. Count grades
    print("\nGrade Count:")
    print("A:", grade_A)
    print("B:", grade_B)
    print("C:", grade_C)
    print("D:", grade_D)
    print("F:", grade_F)

    # 6. Search student
    search_name = input("\nEnter student name to search: ")

    if search_name in students:
        print("Student Found")
        print("Name:", search_name)
        print("Marks:", students[search_name])
    else:
        print("Student not found.")
```

```
30      # Assign grade
```

```
Common odd numbers:
PS C:\Users\Shubham M. Dande\python\assignment8> python q5.py
Student Details:

Name: Amit
Marks: [75, 82, 68]
Total: 225
Average: 75.0
Grade: B

Name: Neha
Marks: [91, 88, 95]
Total: 274
Average: 91.33333333333333
Grade: A

Name: Rahul
Marks: [45, 52, 48]
Total: 145
Average: 48.333333333333336
Grade: F

Students who passed in all subjects:
Amit
Neha
Rahul

Student with highest average:
Neha
Average: 91.33333333333333

Grade Count:
A: 1
B: 1
C: 0
D: 0
F: 1

Enter student name to search: Neha
Student Found
Name: Neha
Marks: [91, 88, 95]
PS C:\Users\Shubham M. Dande\python\assignment8>
```

Q6.
Create a menu-driven Python program using user-defined functions.
The menu should contain:
1. Check Prime Number
2. Check Palindrome
3. Find Factorial
4. Generate Fibonacci Series
5. Exit
Create a separate function for each operation.

ASSIGNMENT8 > q6.py > ...

```python
# Function to check prime
def check_prime(num):
    if num <= 1:
        return False

    for i in range(2, num):
        if num % i == 0:
            return False

    return True


# Function to check palindrome
def check_palindrome(text):
    if text == text[::-1]:
        return True
    else:
        return False


# Function to find factorial
def factorial(num):
    result = 1

    for i in range(1, num + 1):
        result = result * i

    return result


# Function to generate Fibonacci series
def fibonacci(n):
    a = 0
    b = 1

    for i in range(n):
        print(a, end=" ")
        a, b = b, a + b

    print()


# Menu
while True:

    print("\n----- MENU -----")
    print("1. Check Prime Number")
    print("2. Check Palindrome")
    print("3. Find Factorial")
    print("4. Generate Fibonacci Series")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

```

```
    if choice == 1:
        num = int(input("Enter a number: "))

        if check_prime(num):
            print("Prime Number")
        else:
            print("Not a Prime Number")

    elif choice == 2:
        text = input("Enter a string: ")

        if check_palindrome(text):
            print("Palindrome")
        else:
            print("Not a Palindrome")

    elif choice == 3:
        num = int(input("Enter a number: "))

        if num < 0:
            print("Factorial does not exist for negative numbers.")
        else:
            print("Factorial:", factorial(num))

    elif choice == 4:
        n = int(input("Enter number of terms: "))

        print("Fibonacci Series:")
        fibonacci(n)

    elif choice == 5:
        print("Program ended.")
        break

    else:
        print("Invalid choice.")
```

Output:

```
PS C:\Users\Shubham M. Dande\python\assignment8> python q6.py

----- MENU -----
1. Check Prime Number
2. Check Palindrome
3. Find Factorial
4. Generate Fibonacci Series
5. Exit
Enter your choice: 1
Enter a number: 2
Prime Number

----- MENU -----
1. Check Prime Number
2. Check Palindrome
3. Find Factorial
4. Generate Fibonacci Series
5. Exit
Enter your choice: 2
Enter a string: nayan
Palindrome

----- MENU -----
1. Check Prime Number
2. Check Palindrome
3. Find Factorial
4. Generate Fibonacci Series
5. Exit
Enter your choice: 3
Enter a number: 56
Factorial: 710998587804863451854045647463724949736497978881168458687447040000000000000

----- MENU -----
1. Check Prime Number
2. Check Palindrome
3. Find Factorial
4. Generate Fibonacci Series
5. Exit
Enter your choice: 4
Enter number of terms: 3
Fibonacci Series:
0 1 1

----- MENU -----
1. Check Prime Number
2. Check Palindrome
3. Find Factorial
4. Generate Fibonacci Series
5. Exit
Enter your choice: 5
Program ended.
PS C:\Users\Shubham M. Dande\python\assignment8>
```

Q7.
Create a class BankAccount with the following attributes:
account_number
customer_name
balance
Create methods:
deposit()
withdraw()
check_balance()
display_details()
Apply the following conditions:

- Deposit amount must be greater than zero.
- Withdrawal amount must not exceed the balance.
- Display an appropriate message for insufficient balance.
- Allow multiple transactions using a loop.
- Provide a menu:

1. Deposit
2. Withdraw
3. Check Balance
4. Account Details

ASSIGNMENT8 > q7.py > ...

```python
class BankAccount:

    def __init__(self, account_number, customer_name, balance):
        self.account_number = account_number
        self.customer_name = customer_name
        self.balance = balance

    # Deposit method
    def deposit(self, amount):

        if amount > 0:
            self.balance += amount
            print("Amount deposited successfully.")
            print("Deposited amount:", amount)
        else:
            print("Deposit amount must be greater than zero.")

    # Withdraw method
    def withdraw(self, amount):

        if amount <= 0:
            print("Withdrawal amount must be greater than zero.")

        elif amount > self.balance:
            print("Insufficient balance.")

        else:
            self.balance -= amount
            print("Amount withdrawn successfully.")
            print("Withdrawn amount:", amount)

    # Check balance
    def check_balance(self):
        print("Current balance:", self.balance)

    # Display account details
    def display_details(self):
        print("\nAccount Details")
        print("Account Number:", self.account_number)
        print("Customer Name:", self.customer_name)
        print("Balance:", self.balance)


# Create account
account = BankAccount("ACC101", "Shubham", 10000)

# Menu
while True:

    print("\n----- BANK MENU -----")
    print("1. Deposit")
    print("2. Withdraw")
    print("3. Check Balance")
    print("4. Account Details")
    print("5. Exit")
```

```
    choice = int(input("Enter your choice: "))

    if choice == 1:
        amount = float(input("Enter deposit amount: "))
        account.deposit(amount)

    elif choice == 2:
        amount = float(input("Enter withdrawal amount: "))
        account.withdraw(amount)

    elif choice == 3:
        account.check_balance()

    elif choice == 4:
        account.display_details()

    elif choice == 5:
        print("Thank you for using the bank system.")
        break

    else:
        print("Invalid choice.")
```

Output:

```
PS C:\Users\Shubham M. Dande\python\assignment8> python q7.py

----- BANK MENU -----
1. Deposit
2. Withdraw
3. Check Balance
4. Account Details
5. Exit
Enter your choice: 1
Enter deposit amount: 2000
Amount deposited successfully.
Deposited amount: 2000.0

----- BANK MENU -----
1. Deposit
2. Withdraw
3. Check Balance
4. Account Details
5. Exit
Enter your choice: 2
Enter withdrawal amount: 1000
Amount withdrawn successfully.
Withdrawn amount: 1000.0

----- BANK MENU -----
1. Deposit
2. Withdraw
3. Check Balance
4. Account Details
5. Exit
Enter your choice: 3
Current balance: 11000.0

----- BANK MENU -----
1. Deposit
2. Withdraw
3. Check Balance
4. Account Details
5. Exit
Enter your choice: 4

Account Details
Account Number: ACC101
Customer Name: Shubham
Balance: 11000.0

----- BANK MENU -----
1. Deposit
2. Withdraw
3. Check Balance
4. Account Details
5. Exit
Enter your choice:
```

Q8.
Write a Python program using the re module to validate user input.
Validate:

1. Mobile number
2. Email address
3. Password
4. PIN code
5. Username

Use regular expressions and conditional statements to display whether each input is Valid or Invalid.
For the password, enforce:

- At least 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one digit
- At least one special character

ASSIGNMENT8 > q8.py > ...

```python
import re

# Mobile number
mobile = input("Enter mobile number: ")

mobile_pattern = r"^[0-9]{10}$"

if re.match(mobile_pattern, mobile):
    print("Mobile Number: Valid")
else:
    print("Mobile Number: Invalid")


# Email
email = input("Enter email address: ")

email_pattern = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"

if re.match(email_pattern, email):
    print("Email: Valid")
else:
    print("Email: Invalid")


# Password
password = input("Enter password: ")

password_pattern = r"^(?=.*[A-Z])(?=.*[a-z])(?=.*[0-9])(?=.*[!@#$%^&*]).{8,}$"

if re.match(password_pattern, password):
    print("Password: Valid")
else:
```

```python
if re.match(password_pattern, password):
    print("Password: Valid")
else:
    print("Password: Invalid")


# PIN
pin = input("Enter PIN code: ")

pin_pattern = r"^[0-9]{4}$"

if re.match(pin_pattern, pin):
    print("PIN: Valid")
else:
    print("PIN: Invalid")


# Username
username = input("Enter username: ")

username_pattern = r"^[A-Za-z0-9_]{3,15}$"

if re.match(username_pattern, username):
    print("Username: Valid")
else:
    print("Username: Invalid")
```

```
Thank you for using the bank system.
PS C:\Users\Shubham M. Dande\python\assignment8> python q8.py
Enter mobile number: 9764961655
Mobile Number: Valid
Enter email address: shubhamdande@abn.com
Email: Valid
Enter password: shubham
Password: Invalid
Enter PIN code: 123456
PIN: Invalid
Enter username: shubham
Username: Valid
PS C:\Users\Shubham M. Dande\python\assignment8>
```

ASSIGNMENT NO : 9

Write a function apply_discount(price, discount_percentage) where:

- An assert statement checks that discount_percentage is strictly between 0 and 100.
- Another assert statement checks that the final price is never negative or higher than the original price.
- Test your function with a valid discount (e.g., 20%) and an invalid discount (e.g., 120%) to show the traceback.

ASSIGNMENT8 > ASSIGNMENT9 > q1.py > ...

```python
def apply_discount(price, discount_percentage):

    # Check discount is strictly between 0 and 100
    assert 0 < discount_percentage < 100, \
        "Discount percentage must be between 0 and 100."

    final_price = price - (price * discount_percentage / 100)

    # Check final price
    assert 0 <= final_price <= price, \
        "Final price must not be negative or greater than original price."

    return final_price


# Valid discount
price = 1000
discount = 20

print("Original Price:", price)
print("Discount:", discount, "%")
print("Final Price:", apply_discount(price, discount))


# Invalid discount
price = 1000
discount = 120

print("\nTesting invalid discount...")
print(apply_discount(price, discount))
```

```
[Running] python -u "c:\Users\Shubham M. Dande\python\ASSIGNMENT8\ASSIGNMENT9\q1.py"
Original Price: 1000
Discount: 20 %
Final Price: 800.0

Testing invalid discount...
Traceback (most recent call last):
  File "c:\Users\Shubham M. Dande\python\ASSIGNMENT8\ASSIGNMENT9\q1.py", line 30, in <module>
    print(apply_discount(price, discount))
  File "c:\Users\Shubham M. Dande\python\ASSIGNMENT8\ASSIGNMENT9\q1.py", line 4, in apply_discount
    assert 0 < discount_percentage < 100, \
AssertionError: Discount percentage must be between 0 and 100.
```

Create a program that prompts the user to enter two numbers (A and B) and outputs the result of A / B. Wrap the execution inside a try block and implement specific except clauses to handle.

- ValueError: Handled when a user inputs a non-numeric string (e.g., "abc").

- ZeroDivisionError: Handled when B is zero.

ASSIGNMENT8 > ASSIGNMENT9 > q2.py > ...

```python
try:
    A = float(input("Enter number A: "))
    B = float(input("Enter number B: "))

    result = A / B

    print("Result:", result)

except ValueError:
    print("Error: Please enter numeric values only.")

except ZeroDivisionError:
    print("Error: Cannot divide by zero.")
```

```
PS C:\Users\Shubham M. Dande\python\assignment8> cd assignment9
PS C:\Users\Shubham M. Dande\python\assignment8\assignment9> python q2.py
Enter number A: 5
Enter number B: 0
Error: Cannot divide by zero.
PS C:\Users\Shubham M. Dande\python\assignment8\assignment9>
```

Write a program that requests a filename from the user and attempts to open it in read-only mode.

- Intercept FileNotFoundError if the file doesn't exist.
- Intercept PermissionError if the user lacks the credentials to open the file

ASSIGNMENT8 > ASSIGNMENT9 > q3.py > ...

```python
filename = input("Enter filename: ")

try:
    with open(filename, "r") as file:
        content = file.read()

    print("\nFile Content:")
    print(content)

except FileNotFoundError:
    print("Error: File does not exist.")

except PermissionError:
    print("Error: You do not have permission to open this file.")
```

```
Error: Cannot divide by zero.
PS C:\Users\Shubham M. Dande\python\assignment8\assignment9> python q3.py
Enter filename: log.txt
Error: File does not exist.
PS C:\Users\Shubham M. Dande\python\assignment8\assignment9>
```

Create a class named Countdown.

- The constructor _init_(self, start) should accept a starting integer.
- Implement _iter_(self) to return the iterator object itself.
- Implement _next_(self) to return numbers counting down to 0.
- Once it hits 0, it should raise a StopIteration exception to halt the loop.

ASSIGNMENT8 > ASSIGNMENT9 > q4.py > ...

```python
class Countdown:

    def __init__(self, start):
        self.start = start

    def __iter__(self):
        return self

    def __next__(self):

        if self.start < 0:
            raise StopIteration

        current = self.start
        self.start -= 1

        return current


# Create object
countdown = Countdown(5)

# Display countdown
for number in countdown:
    print(number)
```

```
[Running] python -u "c:\Users\Shubham M. Dande\python\ASSIGNMENT8\ASSIGNMENT9\q4.py"
5
4
3
2
1
0

[Done] exited with code=0 in 0.229 seconds
```

Write a generator function named fibonacci_gen(limit).

- It should calculate and yield Fibonacci numbers one by one.
- The generator must stop yielding once a number exceeds the specified limit.

ASSIGNMENT8 > ASSIGNMENT9 > q5.py > ...

```python
def fibonacci_gen(limit):

    a = 0
    b = 1

    while a <= limit:
        yield a

        a, b = b, a + b


# Take limit from user
limit = int(input("Enter the limit: "))

print("Fibonacci numbers:")

for number in fibonacci_gen(limit):
    print(number, end=" ")
```

```
Error: File does not exist.
PS C:\Users\Shubham M. Dande\python\assignment8\assignment9> python q5.py
Enter the limit: 5
Fibonacci numbers:
0 1 1 2 3 5
PS C:\Users\Shubham M. Dande\python\assignment8\assignment9>
```